Computer Vision Group Project

ARI3129

Assignment Report

Daniel Pace, Amy Spiteri, Ellyn Rose Debrincat, Joachim Grech

January 29, 2026

Contents

GitHub repository link: `https://github.com/spiteriamy/ari3129-cv-group-project`

1 Introduction

The Aim of this project was to implement and evaluate various object detection algorithms on a custom dataset. The project was split up into the following sections:

- Building a dataset:
    1. Image collection,
    2. Image cleaning,
    3. Image annotation,
    4. Data analysis, and
    5. Data preprocessing.
- Object detection/ localization:
    1. Implementation of two object detectors to detect sign type,
    2. Implementation of one object detector to detect a specific sign attribute, and
    3. Evaluation of the trained detectors, including metrics and visualizations.
    4. Analysis of input image, supporting the maintenance and monitoring of traffic signs.

The following steps were taken to ensure an equal and fair distribution of work among all group members:

- For task 1, each group member contributed a similar number of annotated images to the dataset.
- For task 2, each team member was assigned one YOLO-based detector and one COCO-based detector to implement and evaluate.

2 Background on the techniques used

2.1 Faster R-CNN

Faster R-CNN [2] represents a significant advancement in object detection by unifying the region proposal step with the detection network, thereby eliminating the computational bottleneck of external algorithms like Selective Search used in R-CNN [4] and Fast R-CNN [3].

The core innovation is the Region Proposal Network (RPN), a fully convolutional network that shares features with the detection head. The RPN slides a window over the convolutional feature map generated by a backbone such as ResNet [5] to simultaneously predict object bounds and objectness scores at each position. To handle multi-scale detection, it utilizes anchor boxes of various scales and aspect ratios. The entire system is trained end-to-end using a multi-task loss function that combines classification and bounding box regression objectives.

2.2 RF-DETR

2.3 RetinaNet

2.4 EfficientDet

EfficientDet [**?**] is an object detection architecture that prioritizes efficiency and scalability. It builds upon the EfficientNet [**?**] backbone and introduces two key innovations: the Weighted Bi-directional Feature Pyramid Network (BiFPN) and a compound scaling method.

The BiFPN allows for easy and fast multi-scale feature fusion by introducing learnable weights to learn the importance of different input features significantly. Unlike traditional FPNs, it incorporates top-down and bottom-up pathways with cross-scale connections to enable more efficient information flow. Furthermore, EfficientDet utilizes a compound scaling method that uniformly scales the resolution, depth, and width for all backbone, feature network, and box/class prediction networks, ensuring optimal performance across a wide range of resource constraints.

2.5 YOLO (You Only Look Once)

YOLO [6] fundamentally changed the approach to object detection by framing it as a single regression problem, rather than a classification task applied to region proposals. Unlike two-stage detectors like Faster R-CNN, which first generate potential regions and then classify them, YOLO processes the full image in a single pass through the neural network.

The architecture divides the input image into an $S \times S$ grid. If the centre of an object falls into a grid cell, that cell is responsible for detecting the object. Each grid cell predicts B bounding boxes and confidence scores for those boxes, along with C class probabilities. This unified architecture allows for extremely fast inference speeds suitable for real-time applications. While early versions (YOLOv1-v3) sometimes struggled with small objects compared to region-based methods, modern iterations (such as the YOLOv8-v12 models used in this project) have incorporated feature pyramids and advanced augmentation techniques to significantly close this gap.

3 Data Preparation

To prepare the dataset for training, the following steps were taken:

1. The images were collected manually from diverse locations on the island, and at different times of the day. The goal was to capture a solid variety of sign conditions, angles, and lighting scenarios.

2. The collected images were then cleaned to remove any duplicates, after which, any recognizable faces or licence plates were obscured to ensure privacy. This was done both manually and using automated tools that can be found in the 'Annotation Process' folder of the GitHub repository [1] (all automated outputs were manually verified before moving onto the next step).

3. The images were then annotated manually using LabelStudio as instructed.

4. Once all images were annotated by each member, the provided `MemberMerger.py` was used to merge every team member's individual annotation JSON files and image ZIP files into one single merged dataset. The output from this step (`merged_input.json` and `merged_images.zip`) was then uploaded to the GitHub repository and can be found in the subdirectory `Dataset/`.

5. The merged Label Studio JSON annotations were then converted into a COCO-formatted dataset by using the provided `LS2COCO.ipynb`. This step produced five COCO-format annotation files, which were uploaded to the GitHub repository and can be found in the `Dataset/COCO-Format Annotations/` subdirectory.

6. The dataset was then split into Train/Validation/Test subsets by using `LS2COCO.ipynb`. A 70%/15%/15% split was chosen to maximize the training data while still retaining sufficient validation and test sets, given the small dataset size. This resulted in a dataset of 485 train images, 111 validation images, and 119 test images.

7. The dataset splitting process was repeated for all attributes and for both YOLO and COCO-based formats.

8. The split dataset was then zipped and uploaded to the shared repository folder (`Dataset/Train-Test-Val Split/`) for easy access by all group members.

3.1 Dataset Class Distribution

Each team member tried to collect a balanced amount of images per class as possible. However, due to some types of signs being less common than others, some imbalances appeared in the final dataset.

The class distribution for the Sign Type (as illustrated in Figure 1) is moderately imbalanced, with No Entry (One Way) signs being the most frequent class (26.19% of all signs). Pedestrian Crossing (19.17%), Stop (18.22%), and Blind-Spot Mirror (16.6%) have similar representation. However, Roundabout Ahead

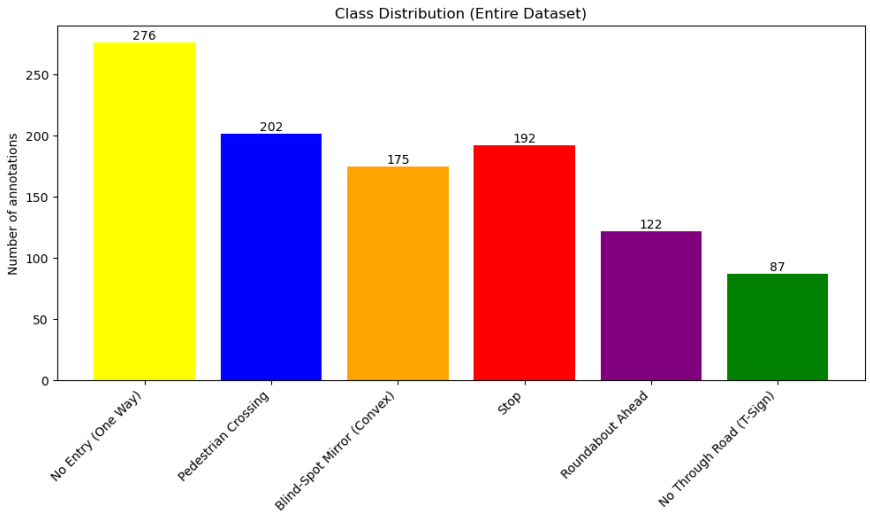

Figure 1: Class Distribution of Sign Types

(11.57%) and No Through Road (8.25%) signs appear less frequently. The distribution can be seen in the bar chart below.

The Sign Shape Type attribute shows a noticeable class imbalance. Circular signs made up nearly half of the dataset (48.58%), followed by square (20.49%) and octagonal (17.55%) signs, which are moderately represented. Triangular signs are significantly less frequent (10.63%), while the damaged class is severely underrepresented (2.75%). This imbalance reflects the real-world frequency of the traffic signs that were encountered during image collection, where circular signs (such as No Entry signs, which were also the most common sign type) were more commonly observed than other sign shapes.

A very large imbalance could also be seen in the Mounting Type attribute, with 87% of all signs being pole-mounted, while only 13% were wall-mounted. This is due to the fact that the majority of signs found across the island were pole-mounted, and wall-mounted signs were much rarer. Similarly, the class distribution for the Sign Condition attribute was also greatly imbalanced. Out of all signs, only 3.8% were labelled as heavily damaged, while 67.08% of signs were in good condition, and 29.13% were weathered.

The Viewing Angle attribute was the most balanced out of all attributes, due to the fact that each sign was photographed from all three viewing angles. 39.75% of all signs were captured from the front, 30.46% from the back, and 29.79% from the side. Perfect balance was not achieved due to signs that also appeared in the background, which were captured from different angles.

4 Implementation of the object detectors

4.1 COCO-based Detectors

The implementation of the different COCO-based detectors varied slightly between models due to their unique architectures and requirements, and having been implemented by different group members. In general, the aim was to match the outputs of the YOLO-based detectors in terms of training performance metrics. The main metric we aimed to optimize was the mAP at IoU=0.5 and at 0.5:0.95. The following COCO-based detectors were implemented:

4.1.1 EfficientDet

The EfficientDet architecture, utilizing the `tf_efficientdet_d0` backbone, was chosen for its optimal trade-off between detection accuracy and resource efficiency. This model integrates an EfficientNet backbone with a BiFPN feature fusion network, allowing it to learn multi-scale feature representations effectively. Pre-trained on ImageNet, the model was fine-tuned to detect the six specific road sign classes in the dataset.

Dataset and Preprocessing A custom `EfficientDetDataset` class handles data loading and transformation. Images are read via OpenCV, converted to RGB, and resized to 512×512 pixels while preserving aspect ratio through padding. Annotations are parsed from the COCO-format JSON, with bounding boxes normalized to the model's required format. Normalization is applied using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).

Augmentation To mitigate overfitting on the small dataset, the `Albumentations` library was employed. Augmentations include horizontal flipping ($p = 0.5$), shift-scale-rotate transformations, random brightness/contrast adjustments, and RGB shift, ensuring the model is robust to varying environmental conditions.

Training Setup Training was conducted using the `DetBenchTrain` module with the AdamW optimizer ($lr = 1e-4$) and a batch size of 4 to fit within GPU constraints. A class balancing strategy was implemented where minority classes were oversampled to match the frequency of the majority class, preventing bias toward frequent sign types. The model heads were fine-tuned while keeping the backbone frozen initially to stabilize convergence.

4.1.2 Faster R-CNN

The Faster R-CNN implementation was based on the pre-trained `fasterrcnn_resnet50_fpn_v2` model from torchvision, using the FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT weights as a starting point. The model was fine-tuned on the custom traffic sign dataset with 6 sign classes plus a background class.

Model Architecture

The base model utilizes a ResNet-50 backbone with Feature Pyramid Network (FPN) for multi-scale feature extraction. The region of interest (RoI) heads were modified to accommodate the custom number of classes by replacing the box predictor with a FastRCNNPredictor configured for 7 output classes.

Dataset Configuration

The training and validation datasets were loaded using PyTorch's CocoDetection class with images transformed to tensors. A custom anchor generator was configured with five anchor sizes (16, 32, 64, 128, 256 pixels) and three aspect ratios (0.5, 1.0, 2.0) to better detect signs of varying sizes. The training used a batch size of 6 with pin memory enabled for faster GPU data transfer.

Training Configuration

The model was trained for up to 100 epochs with the following hyperparameters:

- Learning rate: 0.005
- Momentum: 0.9
- Weight decay: 0.0005
- Optimizer: SGD
- Learning rate scheduler: StepLR (step size=25, gamma=0.1)
- Early stopping patience: 10 epochs

Loss Components

The total training loss was composed of four components: box regression loss, RPN box regression loss, classifier loss, and objectness loss. For logging purposes and comparison with YOLO results, these were grouped into box loss (loss_box_reg + loss_rpn_box_reg) and classification loss (loss_classifier + loss_objectness).

Evaluation Metrics

Validation was performed using COCO evaluation metrics, specifically tracking mAP@IoU=0.50 and mAP@IoU=0.50:0.95. The model's predictions were converted to COCO result format and evaluated using the pycocotools library. Early stopping was implemented based on mAP@IoU=0.50:0.95 to prevent overfitting.

Logging and Monitoring

Training progress was monitored using TensorBoard for real-time visualization of loss curves and metrics. Additionally, a CSV file in YOLO-compatible format was generated containing epoch-wise metrics including training/validation losses, mAP scores, recall, and learning rates. The model was saved periodically every 10 epochs, with the best performing model (based on validation mAP) saved separately.

The implementation was executed on hardware consisting of an AMD Ryzen 9 5950X CPU (16 cores/32 threads), 64 GB RAM, and an NVIDIA GeForce RTX 5070 Ti GPU with 16 GB VRAM, using CUDA 13.1 for acceleration.

4.1.3 RetinaNet

4.1.4 RF-DETR

4.2 YOLO-based Detectors

The implementation of different YOLO versions (YOLOv8, YOLOv11, and YOLOv12) followed a consistent methodology across all group members, with variations primarily in model size selection and hardware-specific optimizations. This section describes the common approach and highlights significant deviations.

Common Implementation Framework

All YOLO implementations utilized the Ultralytics library and followed a standardized workflow consisting of dataset preparation, model training, and evaluation phases.

Dataset Preparation

The dataset preparation phase was consistent across all implementations. Images were extracted from a compressed archive (`images.zip`) into the YOLO dataset directory structure. The `data.yaml` configuration file was programmatically updated to reflect absolute paths for the training, validation, and test splits on each user's system. This ensured portability across different development environments while maintaining consistent dataset access patterns.

Model Selection and Training Configuration

Different YOLO versions and model sizes were employed:

- YOLOv8: Nano variant (`yolov8n.pt`) for baseline experiments
- YOLOv11: Small variant (`yolo11s.pt`) for balanced performance
- YOLOv12: Multiple variants tested (Large, Medium, Small) to evaluate size-performance tradeoffs

The standard training configuration shared across implementations included:

- Epochs: 100 with early stopping patience of 20
- Image size: 640×640 pixels (default)
- Device: Automatic GPU selection when available, CPU fallback otherwise
- Checkpoint saving: Every 10 epochs

- Task: Object detection (`task='detect'`)
- Project output: Organized under `runs/` directory

Significant Deviations

Hardware-Constrained Optimization One implementation utilized reduced batch size (8) and image resolution (512×512) due to hardware memory limitations, demonstrating adaptive configuration for resource-constrained environments.

Advanced Optimizer Configuration An alternative implementation experimented with the Adam optimizer instead of the default SGD, along with disk caching (`cache='disk'`) and increased worker threads (`workers=8`) to optimize I/O performance.

Automated Batch Sizing The YOLOv12 experiments utilized automatic batch sizing (`batch=-1`) targeting 60% GPU utilization, allowing the framework to dynamically determine optimal batch sizes based on available VRAM.

Data Augmentation Experiments Extended experiments with YOLOv12 included systematic comparison of default training versus augmented training with modified hyperparameters:

- Horizontal/vertical flipping disabled (`fliplr=0.0, flipud=0.0`) to preserve sign orientation
- Mosaic augmentation maintained at full strength (`mosaic=1.0`)
- Mixup augmentation introduced (`mixup=0.1`)
- Rotation augmentation enabled (`degrees=10.0`)
- Scale variation applied (`scale=0.5`)

These augmentation strategies were designed specifically for traffic sign detection, where orientation preservation is critical but lighting and scale variations are beneficial.

Monitoring and Logging

All implementations enabled TensorBoard logging for real-time training visualization. Training metrics including loss curves, mAP scores, precision, and recall were logged automatically to CSV files in YOLO's standard format, facilitating cross-model comparison and performance analysis.

5 Evaluation of the object detectors

5.1 COCO-based Detectors

5.1.1 EfficientDet

The EfficientDet model was evaluated using a classification-based approach on ground truth crops due to limited training resources and time constraints. The model achieved an overall validation accuracy of 44.12% after 30 epochs.

The per-class accuracy breakdown highlights significant performance variance:

- No Entry: 91.67% (Highest accuracy)
- Pedestrian Crossing: 66.67%
- Blind Spot Mirror: 34.62%
- Stop: 33.33%

- No Through Road: 0.00%
- Roundabout Ahead: 0.00%

These results indicate that while the model successfully learned features for distinct signs like 'No Entry', it struggled significantly with others, likely due to class imbalance or insufficient training data for those specific categories.

5.1.2 Faster R-CNN

The Faster R-CNN model, utilizing a ResNet-50 FPN V2 backbone, was trained for 58 epochs using the SGD optimizer with an initial learning rate of 0.005. The model was evaluated using standard COCO metrics on the full validation set.

The model achieved the following peak performance metrics:

- mAP@.50: 71.45%
- mAP@.50:.95: 59.46%
- Recall: 68.83%

Analysis of the results indicates strong localization capabilities, though a performance gap persists between large and small objects. As shown in Figure 2 (right), the bounding box accuracy (mAP@.50:.95) continued to improve until approximately epoch 46, even after the classification metrics began to plateau.

A notable observation in the training dynamics is the divergence in classification loss illustrated in Figure 2 (left). While the training classification loss continued to decrease, the validation classification loss exhibited a gradual upward trend after epoch 4. This phenomenon suggests a degree of overfitting, where the model becomes over-confident in the specific categorical features of the training set, leading to a penalty in cross-entropy loss on unseen validation data. However, since the mAP scores remained stable or improved during this period, the model's actual predictive accuracy remained robust, indicating that the overfitting was primarily restricted to loss confidence rather than categorical misclassification. The final combined validation loss settled at 0.2852.

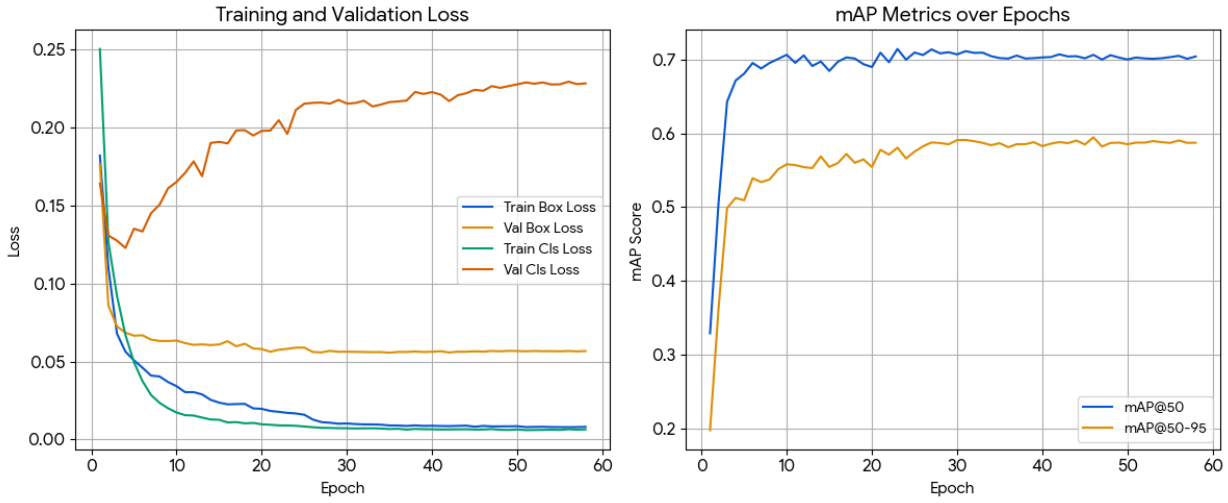

Figure 2: Faster R-CNN Training Performance Metrics.

5.1.3 RetinaNet

5.1.4 RF-DETR

5.2 YOLO-based Detectors

5.2.1 YOLOv8

5.2.2 YOLOv11

5.2.3 YOLOv12

The YOLOv12 model was trained for 100 epochs using the Ultralytics framework. As a state-of-the-art one-stage detector, YOLOv12 utilizes an optimized backbone and neck architecture designed to maximize feature extraction efficiency while maintaining high inference speeds.

The model achieved the following peak performance metrics on the validation set:

- mAP@.50: 81.72%
- mAP@.50:.95: 70.45%
- Precision: 90.02%
- Recall: 79.78%

Analysis of the training results in Figure 3 shows a highly efficient learning curve, with the model surpassing 50% mAP within the first few epochs. The model maintained a consistent, stable trajectory, for the validation classification loss (L_{val_cls}) eventually settling at 0.686. This suggests that the YOLOv12 architecture, likely aided by Distribution Focal Loss (DFL) and advanced data augmentation, is resilient to categorical overfitting on this specific dataset.

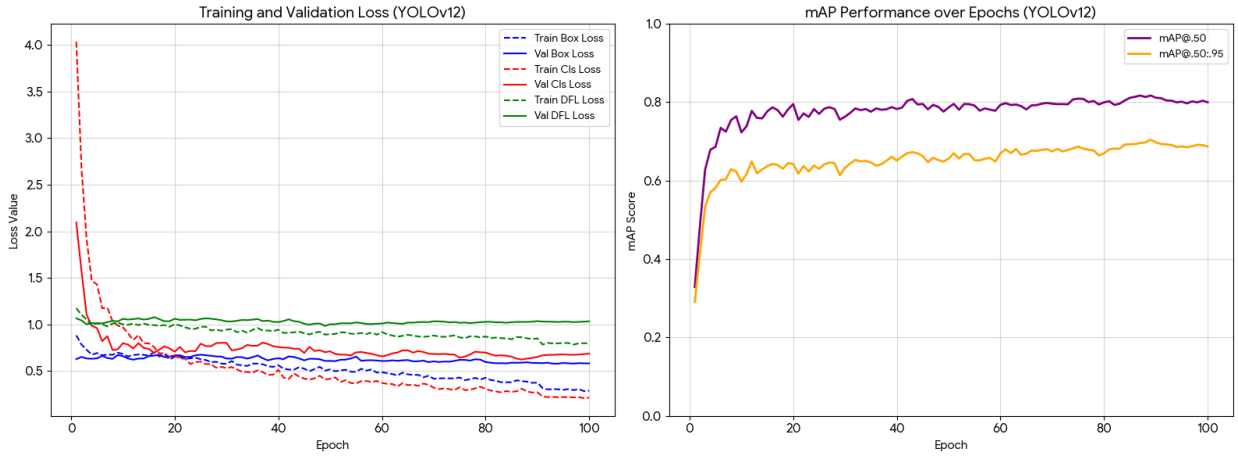

Figure 3: YOLOv12 Training Metrics.

5.3 Comparison between the different detectors

5.3.1 COCO-based vs YOLO-based Detectors

5.3.2 Sign Attribute Comparison

5.3.3 All Detectors together

6 References and List of Resources Used

[1] A. Spiteri, D. Pace, E. R. Debrincat, and J. Grech, "Annotation Process Tools," GitHub repository, spiteriamy/ari3129-cv-group-project, Jan. 2026. [Online]. Available: `https://github.com/spiteriamy/ari3129-cv-group-project/tree/main/Annotation%20Process`

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[3] R. Girshick, "Fast R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1440–1448.

[4] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Columbus, OH, USA, 2014, pp. 580–587.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.